

Product Requirements Document

Semantic Merge Queue & Model-Driven E2E Testing

Document Status: Draft / Review

Target Platform: Buildkite, Azure Container Apps (ACA)

Core Tech Stack: .NET Aspire, Claude Code CLI, Claude 3 Haiku, GitHub

1. Executive Summary

As our distributed systems scale, traditional End-to-End (E2E) testing has become a bottleneck. Hardcoded UI scripts are brittle, and running full environment spin-ups on every Pull Request introduces severe delays and high compute costs.

This PRD outlines the implementation of a **Semantic Merge Queue**. By leveraging Buildkite's dynamic pipelines on self-hosted Azure Container Apps (ACA), we will introduce a two-phase CI/CD gate. A fast LLM will semantically analyze git diffs to determine if behavioral testing is required. If necessary, it will dynamically provision an ephemeral .NET Aspire environment and execute model-driven, intent-based testing using the Claude Code CLI before allowing the code to merge into the mainline branch.

2. Objectives & Success Metrics

- **Reduce CI Compute Costs:** Decrease unnecessary full-environment provisions by 60% by filtering out safe refactors, documentation updates, and cosmetic changes.
- **Increase Merge Velocity:** Reduce the median queue wait time for non-breaking PRs by skipping the latency-heavy E2E execution phase.
- **Zero Maintenance Tests:** Eliminate the need for DOM-based selector updates by shifting to intent-based natural language E2E prompts.
- **Maintain Mainline Stability:** Prevent integration regressions by running tests against the combined state of the PR and the main branch (speculative merge commit).

3. System Architecture

The solution relies on a "Bring Your Own Compute" model using Buildkite to orchestrate jobs running securely within our Azure VNet.

Phase 1: The Semantic Gatekeeper (Triage)

When a PR is approved and added to the GitHub Merge Queue, Buildkite intercepts the `merge_group` webhook. It spins up a lightweight agent that extracts the git diff of the speculative merge commit. A fast LLM (e.g., Claude 3 Haiku) processes the diff to detect structural or behavioral shifts. It outputs a deterministic boolean: `requires_e2e`.

Phase 2: Dynamic Pipeline & E2E Execution

If Phase 1 evaluates to `false`, the pipeline passes immediately, merging the PR. If `true`, the Buildkite agent uses `buildkite-agent pipeline upload` to dynamically generate the steps required for the full test. The heavy runner provisions the .NET Aspire environment and launches Claude Code CLI to execute natural-language test cases against the live localhost deployment.

4. Technical Implementation Details

4.1 Infrastructure & Caching (Azure Container Apps)

We will utilize ephemeral ACA Jobs for our Buildkite agents. To mitigate the "cold start" penalties associated with spinning up .NET and Docker environments, the following caching mechanisms must be implemented:

NuGet Packages	Utilize a Premium Azure Files SMB share mounted directly to the ACA Job container (e.g., <code>/mnt/runner-cache/nuget</code>). Buildkite Agent Hooks will configure the <code>NUGET_PACKAGES</code> environment variable to utilize this persistent mount, reducing <code>dotnet restore</code> times to milliseconds.
Docker/Aspire Layers	Configure Azure Container Registry (ACR) as an inline pull-through cache. Using the Buildkite <code>docker-compose</code> plugin, the pipeline will fetch cached layers from ACR via the high-speed Azure backbone network instead of rebuilding from scratch.
Custom Runner Image	Avoid runtime installations. The base image for the ACA Job will be pre-baked with the .NET SDK, Aspire Workloads, Claude Code CLI, and Buildkite Agent binaries.

4.2 Buildkite Dynamic Pipeline Configuration

The core orchestration relies on Buildkite's ability to inject steps at runtime. The initial `pipeline.yml` will only contain the Gatekeeper step:

```
steps:
  - label: ":robot_face: Semantic Triage"
    command: "./scripts/evaluate-diff.sh"
    agents:
      queue: "lightweight-aca"
```

Inside `evaluate-diff.sh`, if the LLM determines testing is required, the script outputs the secondary pipeline:

```
cat <<EOF | buildkite-agent pipeline upload
steps:
  - label: ":test_tube: Agentic E2E via Aspire"
    command: "aspire start & claude-code --prompt 'Verify checkout flow'"
    agents:
      queue: "heavy-aca"
EOF
```

5. Edge Cases & Mitigations

- **False Negatives (Skipping necessary tests):** Deeply nested backend interface changes might not immediately register as "user-facing UI breaks" to the Gatekeeper model. *Mitigation:* The prompt for the Phase 1 LLM must default to `true` (run tests) if confidence is low. Additionally, hardcode specific paths (e.g., core infrastructure DTOs) to automatically bypass the LLM and trigger the tests.
- **Non-Determinism in E2E:** Claude Code might attempt different paths through the application on consecutive runs. *Mitigation:* Provide strict contextual constraints in the test prompt, binding the agent to a specific user journey while allowing flexibility only in structural DOM navigation.
- **Token Rate Limits:** Heavy concurrent PR merges could throttle our Anthropic API tier. *Mitigation:* Implement Buildkite concurrency limits on the ``heavy-aca`` queue to serialize massive E2E runs.

6. Next Steps

1. Provision the Premium Azure File Share and update the ACA Job Bicep templates to mount the volumes.

2. Build and publish the custom "Fat" Buildkite Agent Docker image containing .NET Aspire and Claude CLI to our internal ACR.
3. Draft the Phase 1 LLM prompt (The Semantic Gatekeeper) and run a dry-run analysis on historical closed PRs to tune accuracy and false-negative rates.